



About FLYYY.AI

FLYYY.AI is building the flywheel for enterprise AI initiatives by unifying **data governance, AI governance, and regulatory compliance** into a single intelligent platform.

Enterprises are increasingly embedding AI into products, workflows, and decision-making processes, often using sensitive business and customer data. As adoption grows, organizations face a core governance challenge: understanding **where AI is being used, what systems exist, and how those systems interact with data**.

The issue is not a lack of tools, but a lack of visibility. Engineering, data, AI, and compliance teams often operate in silos, making it difficult to maintain a unified view of the AI landscape. FLYYY.AI addresses this by creating a centralized intelligence layer that discovers, maps, and governs AI usage across the organization, ensuring transparency, control, and compliance across all AI systems.

We are actively looking for interns who are excited to work in this problem space and help build the future of AI governance.

Problem Statement

Policy-to-Evidence Compliance Evaluation

Compliance platform automatically collects raw operational metrics and technical evidence from cloud infrastructure, applications, databases, and AI workloads. However, enterprise compliance is not one-size-fits-all. Every organization maintains its own internal governance rules, often written in unstructured policy documents such as PDFs.

Hardcoding compliance logic into software does not scale because every customer may define different controls, thresholds, exceptions, and operational requirements. The challenge is to convert unstructured policy language into structured, machine-evaluatable rules and compare those rules against live evidence.

This creates a core product problem: organizations need a system that can ingest a policy PDF, extract the relevant controls, evaluate infrastructure evidence against those controls, and present clear compliance verdicts in a user-friendly dashboard.

This creates a fundamental problem:



How can an organization automatically transform policy documents into evaluable compliance rules and determine whether live system evidence satisfies those rules?

Your Challenge

The core task is to **build an end-to-end compliance evaluation web application** that can upload an unstructured policy PDF, extract policy controls into structured rules, accept evidence JSON, evaluate the evidence against the extracted rules, and display the results in a clear dashboard.

The application should demonstrate the complete flow from policy ingestion to rule extraction, evidence evaluation, and audit-ready reporting. The goal is not to build a perfect legal compliance system, but to show strong system design, clean engineering, and a thoughtful approach to translating governance language into practical evaluation logic.

For example, a policy may state that production database servers must maintain CPU utilization below 85% and must always have auto-scaling enabled. Your system should extract those requirements as structured rules and compare them against evidence collected from live infrastructure metrics.

You may create your own mock policy PDF content and mock evidence JSON payload to demonstrate the full evaluation workflow. The mock policy should read like a realistic company policy, using language such as “we maintain,” “our systems must,” and “we follow,” while the evidence JSON may contain sample infrastructure metrics and configuration values needed to test the extracted rules.

The application should include both backend services and a functional frontend UI. Candidates may choose the extraction strategy, rule format, evaluation model, and storage design, but they should clearly document the technical decisions and trade-offs.

The primary focus of the assignment is the **policy-to-evidence evaluation pipeline**: unstructured policy text to structured controls, structured controls to evaluation logic, and evaluation logic to explainable compliance results.

Quick Domain Glossary

Use these terms to understand the compliance evaluation flow:

Policy: An organization’s internal rule document, usually provided as a PDF or DOCX.



Control: A specific rule inside a policy, often identified by a control ID such as ISO-27001-8.6.

Evidence: Raw technical data from live systems or cloud, such as CPU utilization, configuration values, logs, or metrics.

Compliance Verdict: The result of comparing evidence against a control: Compliant if the rule passes, Non-Compliant if it fails.

Minimal User Interface Requirements

Candidates must build a functional web UI that covers the following core user flows.

Screen 1: Policy Upload and Extraction Preview

- **Upload Component:** Provide a drag-and-drop or file selector component to upload a policy PDF document
- **Extracted Rules View:** After upload, display extracted rules in a clean table or card list showing Control ID, Target Metric, and Condition or Threshold.

Screen 2: Compliance Scan and Results Dashboard

- **Evidence Input:** Provide a component to upload or paste an evidence JSON payload representing live infrastructure metrics.
- **Trigger Scan:** Provide a clear button to start the evaluation engine.
- **Results Dashboard:** Display the overall compliance status, asset-level checks, and plain-text audit reasoning for each rule evaluation.

Concrete Input and Output Example

Input: Policy Snippet from PDF

We maintain sufficient production computing capacity to ensure reliable service delivery and to prevent unexpected outages. As part of our internal infrastructure governance rules, we continuously monitor compute resources across our production application and database servers, including CPU utilization and related capacity indicators. Our production application and database servers are required to operate with CPU utilization below 85% under normal operating conditions. We also keep auto-scaling enabled for applicable production workloads so that additional compute capacity can be provisioned automatically when demand increases. We review, document, and remediate any sustained capacity threshold breach or disabled auto-scaling configuration through our established incident and change management processes.



Input: Raw Platform Evidence JSON

```
{ "scan_id": "SCAN-2026-0812", "environment": "production", "assets": [{ "asset_id": "prod-db-server-01", "asset_type": "database_server", "metrics": { "cpu_utilization": 92, "auto_scaling_enabled": true } } ] }
```

- **Expected Overall Status:** Non-Compliant
- **CPU Utilization Check:** Non-Compliant because CPU utilization is 92%, which exceeds the maximum policy threshold of 85%.
- **Auto-Scaling Check:** Compliant because auto-scaling is enabled as required.
- **Audit Reasoning:** The dashboard should explain each result in plain language so that a reviewer can understand why the asset passed or failed.

The objective is to demonstrate this complete flow from **finding AI usage to producing a meaningful AI asset**.

Scope

The objective is not to simply build a PDF parser, keyword extractor, or rule engine that checks a few hardcoded compliance conditions.

Build a solution that demonstrates how unstructured policy language can be **systematically extracted, converted into structured controls, and evaluated against technical evidence** to produce clear compliance verdicts.

The sample environment should be small enough to reproduce and understand, while the application design should be flexible enough to support different policy formats, control types, evidence fields, and evaluation conditions.

You should research the problem, make reasonable assumptions, choose your own discovery approach, and build a working solution.

We value **depth of understanding over breadth of implementation**.

A smaller, well-designed solution that demonstrates strong reasoning, clean engineering, and a clear understanding of the problem is more valuable than a solution that integrates numerous APIs without a clear understanding of **why they were chosen, how they work, or what problem they actually solve**.

Core Requirements

- **Language:** Python
- **Backend:** FastAPI



- **Frontend:** React.js preferred; Next.js or Angular is also acceptable
- **Database:** RDBMS; PostgreSQL preferred
- **Repository:** Public GitHub repository
- **Deployment:** Deployed application link, if possible
- **Agent Framework:** Use an agent or LLM framework such as LangGraph only if it adds clear value to the solution. If an agent-based approach is not required, do not include an agent. It is completely acceptable for the project to be built without any AI agent.
- **System Design:** Include the system architecture/design in the GitHub README. An Excalidraw diagram is preferred.
- **Code Quality:** Follow proper coding standards, project structure, naming conventions, error handling, documentation, and maintainability practices.

What We Evaluate

We are not evaluating the solution based solely on how many features you implement.

We will primarily evaluate:

- **Problem understanding**
- **Depth of technical research**
- **Engineering quality**
- **System design**
- **Quality of implementation**
- **Ability to reason about trade-offs**
- **Handling of edge cases**
- **Clarity of documentation**
- **Ability to explain why a particular approach was chosen**
- **Ability to identify limitations and improve the solution**

We value **depth of understanding over breadth of implementation**.

A smaller, well-designed solution that demonstrates strong reasoning, clean engineering, and a clear understanding of the problem is more valuable than a solution that integrates numerous APIs without a clear explanation of **why they were chosen, how they work, or what problem they actually solve**.